

## Interface Specification ver\_2.0

### 1. 작성 기준

이 문서는 “누가 누구에게 무엇을 보내고, 무엇을 받는가”를 기준으로 인터페이스를 분류한다.

| 항목            | 기준                                                                                                                                     |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------|
| UI 통신         | Customer UI와 Admin UI는 Control Server의 HTTP/WebSocket API를 사용한다.                                                                       |
| 데이터 저장        | Control Server가 DB에 orders, order_item, product, pickup_slot, robot_unit, robot, task, task_event, stocking_item, exception_log를 저장한다. |
| 주문 감지         | Fleet Manager가 Control Server를 polling하여 ORDER_WAIT 주문을 감지한다.                                                                          |
| 입고 감지         | Fleet Manager가 Control Server를 polling하여 REQUESTED stocking_item을 감지한다.                                                                |
| 주문 task 생성/배정 | Fleet Manager의 TaskManager가 task 생성, robot_unit 배정, sequence_no 부여를 담당한다.                                                              |
| 입고 task 생성/배정 | Fleet Manager의 TaskManager가 stocking_item을 읽어 입고 task 4개를 생성한다.                                                                        |
| 경로/교통 판단      | Fleet Manager의 TrafficManager가 PICKY 이동 경로와 구역 점유 가능 여부를 판단한다.                                                                         |
| 로봇 제어         | PICKY와 COBOT은 Control Server와 직접 task를 주고받지 않고 Fleet Manager를 통해 제어된다.                                                                 |

|          |                                                                                               |
|----------|-----------------------------------------------------------------------------------------------|
| 로봇 상태 보고 | Fleet Manager가 Control Server API 로<br>robot/task/order/stocking_item/exce<br>ption 상태를 보고한다. |
| LLM      | Control Server가 LLM 담당 모듈 또<br>는 AI Server LLM에 자연어 명령 파<br>싱을 요청한다.                          |
| Vision   | PICKY/COBOT State Manager 또는<br>Vision Streamer가 AI Server Vision<br>에 직접 요청한다.               |
| 이미지/영상   | Control Server는 이미지/영상 데이터<br>를 중계하지 않는다.                                                     |
| 실시간 UI   | Control Server는 상태 변경 시<br>Customer/Admin UI에 WebSocket으<br>로 push한다.                         |
| ROS2 통신  | Fleet Manager와 State Manager가<br>ROS2 Topic/Action/Service로<br>PICKY/COBOT을 제어한다.             |

## 2. 전체 흐름 요약

| 구분       | 흐름                             | 핵심 역할                            |
|----------|--------------------------------|----------------------------------|
| 고객 주문    | Customer UI → Control Server   | 상품 조회, 주문 생성, 주문 상태 확인, 수령 완료    |
| 관리자 관제   | Admin UI → Control Server      | 로봇/주문/task/재고/픽업슬롯/예외 상태 조회 및 제어 |
| 주문 감지    | Fleet Manager → Control Server | ORDER_WAIT 주문 polling            |
| 주문 작업 생성 | Fleet Manager → Control Server | task 생성, robot_unit 배정, 상태 보고    |

|           |                                              |                                              |
|-----------|----------------------------------------------|----------------------------------------------|
| 입고 명령 생성  | Admin UI → Control Server → LLM              | 자연어 입고 명령 파싱 후 stocking_item 생성              |
| 입고 작업 감지  | Fleet Manager → Control Server               | REQUESTED<br>stocking_item polling           |
| 입고 작업 생성  | Fleet Manager → Control Server               | 입고 task 생성,<br>stocking_item 상태 변경, 입고 완료 보고 |
| 로봇 제어     | Fleet Manager → PICKY/COBOT State Manager    | 이동, 상차, 검수, 하차, 입고, 복귀, 도킹, 충전, 긴급정지 명령      |
| Vision 인식 | PICKY/COBOT State Manager → AI Vision Server | 상품 인식, 검수, 입고 상품 인식, 사람 감지                   |
| UI 실시간 갱신 | Control Server → Customer/Admin UI           | WebSocket 상태 push                            |

### 3. Customer UI ↔ Control Server

고객이 상품을 조회하고 주문하며, 진행 상태를 확인하고 수령 완료까지 처리하는 흐름이다.

| IF ID   | 호출 시점             | 송신자         | 수신자            | 목적         | 요청                | 응답 형태            | 프로토콜     |
|---------|-------------------|-------------|----------------|------------|-------------------|------------------|----------|
| IF-C-01 | 고객이 상품 목록 화면 진입 시 | Customer UI | Control Server | 상품 목록 조회   | GET /api/products | product 목록       | HTTP/TCP |
| IF-C-02 | 고객이 상품 선택/수량 변경 시 | Customer UI | Customer UI    | 장바구니 상태 변경 | 서버 API 없음         | 브라우저 local state | Local    |
| IF-C-03 | 고객이 주문 확정 시       | Customer UI | Control Server | 주문 생성      | POST /api/        | order 상세         | HTTP/TCP |

|         |                     |                |                |                   |                                      |                  |               |
|---------|---------------------|----------------|----------------|-------------------|--------------------------------------|------------------|---------------|
|         |                     |                |                |                   | order<br>s                           |                  |               |
| IF-C-04 | 고객이 주문 현황 확인 시      | Customer UI    | Control Server | 주문 상태 조회          | GET /api/orders/{order_id}           | order 상세         | HTTP/TCP      |
| IF-C-05 | 고객 화면 초기 진입/새 로고침 시 | Customer UI    | Control Server | 고객 화면 전체 상태 조회    | GET /api/customer/status             | products, orders | HTTP/TCP      |
| IF-C-06 | 주문/상품 상태 변경 시 자동    | Control Server | Customer UI    | 고객 화면 실시간 상태 push | WS /api/customer/status              | customer status  | WebSocket/TCP |
| IF-C-07 | 고객이 상품 수령 후         | Customer UI    | Control Server | 수령 완료 처리          | POST /api/orders/{order_id}/complete | completed order  | HTTP/TCP      |

#### IF-C-03 요청 예시

```

1 {
2   "items": [
3     {
4       "product_id": 1,
5       "quantity": 2
6     },
7     {
8       "product_id": 3,
9       "quantity": 1

```

```

10     }
11   ]
12 }

```

#### IF-C-03 처리 원칙

| 항목            | 처리                                          |
|---------------|---------------------------------------------|
| orders        | 생성 후 <code>status=ORDER_WAIT</code>         |
| order_item    | 주문 상품 목록 생성                                 |
| product       | 주문 수량만큼 <code>stock_qty</code> 차감           |
| task          | Control Server가 생성하지 않음                     |
| Fleet Manager | 이후 polling으로 <code>ORDER_WAIT</code> 주문을 감지 |

#### 4. Admin UI ↔ Control Server

관리자가 로봇, 주문, 작업, 재고, 픽업 슬롯, 예외를 조회하고 제어하는 흐름이다.

| IF ID   | 호출 시점           | 송신자            | 수신자            | 목적                 | 요청                                     | 응답 형태                      | 프로토콜          |
|---------|-----------------|----------------|----------------|--------------------|----------------------------------------|----------------------------|---------------|
| IF-A-01 | 관리자 대시보드 진입 시   | Admin UI       | Control Server | 관리자 통합 상태 조회       | <code>GET /api/admin/status</code>     | admin status               | HTTP/TCP      |
| IF-A-02 | 상태 변경 시 자동      | Control Server | Admin UI       | 관리자 화면 실시간 상태 push | <code>WS /api/admin/ws/status</code>   | admin status               | WebSocket/TCP |
| IF-A-03 | 관리자가 긴급 정지 클릭 시 | Admin UI       | Control Server | 전체 로봇 긴급 정지 요청     | <code>POST /api/admin/emergency</code> | <code>{status:"ok"}</code> | HTTP/TCP      |

|         |                     |          |                |                |                                                 |                           |          |
|---------|---------------------|----------|----------------|----------------|-------------------------------------------------|---------------------------|----------|
|         |                     |          |                |                | gency<br>-<br>stop                              |                           |          |
| IF-A-04 | 관리자가 재개 클릭 시        | Admin UI | Control Server | 긴급 정지 해제/재개 요청 | POST<br>/api/admin/resume                       | {status:"ok"}             | HTTP/TCP |
| IF-A-05 | 관리자가 신규 상품 등록 시     | Admin UI | Control Server | 상품 등록          | POST<br>/api/admin/products                     | product                   | HTTP/TCP |
| IF-A-06 | 관리자가 상품 정보 수정 시     | Admin UI | Control Server | 상품 수정          | PATCH<br>/api/admin/products/{product_id}       | product                   | HTTP/TCP |
| IF-A-07 | 관리자가 재고 수량만 수정 시    | Admin UI | Control Server | 재고 수량 수정       | PATCH<br>/api/admin/products/{product_id}/stock | product                   | HTTP/TCP |
| IF-A-08 | 관리자가 자연어 입고 명령 입력 시 | Admin UI | Control Server | LLM 입고 명령 처리   | POST<br>/api/admin/llm/                         | LLM 처리 결과 및 stocking_item | HTTP/TCP |

|         |                    |          |                |                |                                                      |               |          |
|---------|--------------------|----------|----------------|----------------|------------------------------------------------------|---------------|----------|
|         |                    |          |                |                | messa<br>ges                                         |               |          |
| IF-A-09 | 관리자가 예외 처리 완료 시    | Admin UI | Control Server | 예외 resolved 처리 | POST<br>/api/admin/exceptions/{exception_id}/resolve | {status:"ok"} | HTTP/TCP |
| IF-A-10 | 관리자가 픽업 슬롯 상태 변경 시 | Admin UI | Control Server | 픽업 슬롯 상태 수동 변경 | PATCH<br>/api/fleet/pickup-slots/{slot_id}           | {status:"ok"} | HTTP/TCP |

#### IF-A-08 요청 예시

```

1 {
2   "message": "콜라 5개 입고해줘"
3 }

```

#### IF-A-08 응답 예시

```

1 {
2   "result": "ok",
3   "message": "콜라 5개 입고 요청을 생성했습니다.",
4   "action": "STOCKING",
5   "product_id": 1,
6   "product_name": "콜라",
7   "requested_quantity": 5,
8   "stocking_policy": "REQUESTED_QUANTITY",
9   "stocking_item_id": 1,
10  "provider": "llm"
11 }

```

## 5. Control Server ↔ Fleet Manager

Control Server는 DB와 API를 제공한다.

Fleet Manager는 Control Server를 polling하여 주문/입고 요청을 감지하고, task 생성/상태 보고/예외 보고를 수행한다.

### 5-1. Fleet Manager가 Control Server에 조회하는 API

| IF ID   | 호출 시<br>점                   | 송신자           | 수신자            | 목적         | 요청                                      | 응답 형<br>태        | 프로토<br>콜 |
|---------|-----------------------------|---------------|----------------|------------|-----------------------------------------|------------------|----------|
| IF-F-01 | Fleet Manager 시작 또는 재 동기화 시 | Fleet Manager | Control Server | 초기 상태 동기화  | GET /api/fleet/snapshot                 | admin status와 유사 | HTTP/TCP |
| IF-F-02 | Fleet Manager 시작 또는 경로 계산 전 | Fleet Manager | Control Server | zone 좌표 조회 | GET /api/fleet/zones?type={zone_type}   | zone 목록          | HTTP/TCP |
| IF-F-03 | 주기 polling                  | Fleet Manager | Control Server | 대기 주문 조회   | GET /api/fleet/orders?status=ORDER_WAIT | order summary 목록 | HTTP/TCP |
| IF-F-04 | 주문 처리 전                     | Fleet Manager | Control Server | 주문 상세 조회   | GET /api/                               | order 상세         | HTTP/TCP |



|         |            |               |                |                  |                                                       |                  |          |
|---------|------------|---------------|----------------|------------------|-------------------------------------------------------|------------------|----------|
|         |            | er            |                |                  | order<br>s/{or<br>der_i<br>d}                         |                  |          |
| IF-F-05 | 주문 처리 전    | Fleet Manager | Control Server | 기존 task 중복 확인    | GET<br>/api/fleet/orders/{order_id}/tasks             | task 목록          | HTTP/TCP |
| IF-F-06 | 픽업 이동 전    | Fleet Manager | Control Server | 빈 pickup slot 조회 | GET<br>/api/fleet/pickup-slots?<br>status=EMPTY       | pickup slot 목록   | HTTP/TCP |
| IF-F-07 | 주기 polling | Fleet Manager | Control Server | 대기 입고 item 조회    | GET<br>/api/fleet/stocking-items?<br>status=REQUESTED | stocking_item 목록 | HTTP/TCP |
| IF-F-08 | 입고 처리 전    | Fleet Manager | Control Server | 상품/zon           | GET<br>/api/                                          | product 목록       | HTTP/TCP |

|  |  |    |  |            |                                    |  |  |
|--|--|----|--|------------|------------------------------------|--|--|
|  |  | er |  | e 정보<br>조회 | produ<br>cts<br>또는<br>snapsh<br>ot |  |  |
|--|--|----|--|------------|------------------------------------|--|--|

## 5-2. Fleet Manager가 Control Server에 변경 요청하는 API

| IF ID   | 호출 시<br>점                         | 송신자                  | 수신자               | 목적                    | 요청                                                        | 응답 형<br>태             | 프로토<br>콜     |
|---------|-----------------------------------|----------------------|-------------------|-----------------------|-----------------------------------------------------------|-----------------------|--------------|
| IF-F-09 | robot_u<br>nit 배정<br>후            | Fleet<br>Manag<br>er | Control<br>Server | 주문 담<br>당 unit<br>기록  | PATCH<br>/api/<br>fleet<br>/orde<br>rs/{o<br>rder_<br>id} | {stat<br>us:"o<br>k"} | HTTP/<br>TCP |
| IF-F-10 | task 계<br>획 생성<br>후               | Fleet<br>Manag<br>er | Control<br>Server | task 일<br>괄 생성        | POST<br>/api/<br>fleet<br>/task<br>s/bul<br>k             | task_id<br>s          | HTTP/<br>TCP |
| IF-F-11 | task 시<br>작/완<br>료/실<br>패/정지<br>시 | Fleet<br>Manag<br>er | Control<br>Server | task 상<br>태 보고        | PATCH<br>/api/<br>fleet<br>/task<br>s/{ta<br>sk_id<br>}   | {stat<br>us:"o<br>k"} | HTTP/<br>TCP |
| IF-F-12 | task 세<br>부 이벤<br>트 발생<br>시       | Fleet<br>Manag<br>er | Control<br>Server | task_ev<br>ent 기<br>록 | POST<br>/api/<br>fleet<br>/task<br>s/{ta                  | task_ev<br>ent        | HTTP/<br>TCP |

|         |                                    |                      |                   |                                 |                                                                                        |                       |              |
|---------|------------------------------------|----------------------|-------------------|---------------------------------|----------------------------------------------------------------------------------------|-----------------------|--------------|
|         |                                    |                      |                   |                                 | sk_id<br>}/events                                                                      |                       |              |
| IF-F-13 | robot<br>위치/배<br>터리/상<br>태 변경<br>시 | Fleet<br>Manag<br>er | Control<br>Server | robot<br>상태 보<br>고              | PATCH<br>/api/<br>fleet<br>/robo<br>ts/{r<br>obot_<br>id 또<br>는<br>robot<br>_name<br>} | {stat<br>us:"o<br>k"} | HTTP/<br>TCP |
| IF-F-14 | pickup<br>zone 선<br>택 후            | Fleet<br>Manag<br>er | Control<br>Server | 선택된<br>pickup<br>slot 주<br>문 배정 | PATCH<br>/api/<br>fleet<br>/orde<br>rs/{o<br>rder_<br>id}                              | {stat<br>us:"o<br>k"} | HTTP/<br>TCP |
| IF-F-15 | pickup<br>slot 상<br>태 변경<br>시      | Fleet<br>Manag<br>er | Control<br>Server | pickup<br>slot 상<br>태 갱신        | PATCH<br>/api/<br>fleet<br>/pick<br>up-<br>slots<br>/{slo<br>t_id<br>}                 | {stat<br>us:"o<br>k"} | HTTP/<br>TCP |
| IF-F-16 | 예외 발<br>생 시                        | Fleet<br>Manag<br>er | Control<br>Server | 예외 기<br>록                       | POST<br>/api/<br>fleet<br>/exce                                                        | excepti<br>on_id      | HTTP/<br>TCP |

|         |                 |               |                |                     |                                                    |               |          |
|---------|-----------------|---------------|----------------|---------------------|----------------------------------------------------|---------------|----------|
|         |                 |               |                |                     | ptions                                             |               |          |
| IF-F-17 | 입고 item 상태 변경 시 | Fleet Manager | Control Server | stocking_item 상태 변경 | PATCH /api/fleet/stocking-items/{stocking_item_id} | stocking_item | HTTP/TCP |
| IF-F-18 | 입고 적재 완료 후      | Fleet Manager | Control Server | 입고 완료 및 재고 증가       | POST /api/fleet/stocking/complete                  | stock 반영 결과   | HTTP/TCP |

#### 주문 polling 처리 흐름

```

1 1. Fleet Manager가 주기적으로 GET /api/fleet/orders?status=ORDER_WAIT 호출
2 2. TaskManager가 주문별 기존 task 존재 여부 확인
3 3. 기존 task가 있으면 skip
4 4. 기존 task가 없으면 robot_unit 배정
5 5. 주문 item을 product zone/slot 기준 dict로 정규화
6 6. TrafficManager에 상품 후보 dict 전달
7 7. TrafficManager가 가장 가까운 상품 zone 경로 예약
8 8. TaskManager가 선택된 상품에 대한 task 생성
9 9. task 상태 변화는 Fleet Manager가 Control Server에 보고

```

#### 입고 polling 처리 흐름

```

1 1. Admin UI가 자연어 입고 명령 입력
2 2. Control Server/LLM이 stocking_item 생성
3 3. Fleet Manager가 주기적으로 GET /api/fleet/stocking-items?status=REQUESTED 호출
4 4. TaskManager가 stocking_item을 읽어 robot_unit 배정
5 5. TaskManager가 입고 task 4개 생성
6 6. Fleet Manager가 task 상태와 stocking_item 상태를 Control Server에 보고

```

#### task 일괄 생성 요청 예시

```

1 {
2   "tasks": [

```

```

3      {
4          "order_id": 7,
5          "order_item_id": 22,
6          "stocking_item_id": null,
7          "sequence_no": 1,
8          "assigned_robot_name": "PICKY1",
9          "task_type": "MOVE_TO_PRODUCT",
10         "status": "ASSIGNED",
11         "priority": 2,
12         "source_zone_id": 1,
13         "target_zone_id": 6,
14         "result_message": null
15     },
16     {
17         "order_id": 7,
18         "order_item_id": 22,
19         "stocking_item_id": null,
20         "sequence_no": 2,
21         "assigned_robot_name": "COBOT1",
22         "task_type": "SORTING_AND_LOAD",
23         "status": "ASSIGNED",
24         "priority": 2,
25         "source_zone_id": 12,
26         "target_zone_id": 6,
27         "result_message": null
28     }
29 ]
30 }

```

#### 입고 task 생성 예시

```

1  {
2      "tasks": [
3          {
4              "order_id": null,
5              "order_item_id": null,
6              "stocking_item_id": 1,
7              "sequence_no": 1,
8              "assigned_robot_name": "PICKY1",
9              "task_type": "MOVE_TO_STOCK",
10             "status": "ASSIGNED",
11             "priority": 1,
12             "source_zone_id": 1,
13             "target_zone_id": 3,
14             "result_message": null
15         },
16         {
17             "order_id": null,
18             "order_item_id": null,
19             "stocking_item_id": 1,
20             "sequence_no": 2,
21             "assigned_robot_name": "COBOT1",
22             "task_type": "STOCKING_PICK",
23             "status": "ASSIGNED",
24             "priority": 1,
25             "source_zone_id": 4,
26             "target_zone_id": 4,
27             "result_message": null
28         },
29         {
30             "order_id": null,
31             "order_item_id": null,
32             "stocking_item_id": 1,
33             "sequence_no": 3,
34             "assigned_robot_name": "PICKY1",
35             "task_type": "MOVE_TO_STORAGE",

```

```

36     "status": "ASSIGNED",
37     "priority": 1,
38     "source_zone_id": 3,
39     "target_zone_id": 8,
40     "result_message": null
41   },
42   {
43     "order_id": null,
44     "order_item_id": null,
45     "stocking_item_id": 1,
46     "sequence_no": 4,
47     "assigned_robot_name": "COBOT1",
48     "task_type": "STOCKING_PLACE",
49     "status": "ASSIGNED",
50     "priority": 1,
51     "source_zone_id": 4,
52     "target_zone_id": 14,
53     "result_message": null
54   }
55 ]
56 }

```

#### task 상태 보고 요청 예시

```

1 {
2   "current_status": "ASSIGNED",
3   "status": "RUNNING",
4   "assigned_robot_name": "PICKY1",
5   "result_message": "MOVE_TO_PRODUCT started"
6 }

```

#### robot 상태 보고 예시

```

1 {
2   "robot_status": "BUSY",
3   "picky_state": "MOVING_TO_PRODUCT",
4   "cobot_state": null,
5   "current_task_id": 101,
6   "battery_level": 87,
7   "pos_x": 1.25,
8   "pos_y": 3.44,
9   "pos_theta": 0.78
10 }

```

#### 선택된 pickup slot 배정 예시

TrafficManager가 PICKUP\_ZONE\_2 를 선택한 경우 TaskManager는 대응되는 PICKUP\_SLOT\_2 를 주문에 배정한다.

```

1 {
2   "pickup_slot_id": 2
3 }

```

#### 호출:

```

1 PATCH /api/fleet/orders/{order_id}

```

#### 예외 기록 요청 예시

```

1 {
2   "exception_type": "NAVIGATION_FAILED",

```

```

3  "robot_name": "PICKY1",
4  "task_id": 101,
5  "order_id": 7,
6  "detail": "PICKY1 failed to reach PRODUCT_ZONE_3"
7  }

```

#### 입고 완료 요청 예시

```

1  {
2    "task_id": 204,
3    "detected_quantity": 7,
4    "stock_delta": 7,
5    "result_message": "콜라 7개 입고 완료"
6  }

```

## 6. Fleet Manager 내부 클래스 역할

Fleet Manager는 하나의 ROS2 Node 안에서 일반 Python class들을 조립해 사용한다.

```

1  FleetManagerNode
2    ├── ControlServerClient
3    ├── TaskManager
4    ├── TrafficManager
5    ├── RobotCommandGateway
6    └── RobotStateMonitor

```

| 클래스                 | 역할                                                                |
|---------------------|-------------------------------------------------------------------|
| FleetManagerNode    | 유일한 ROS2 Node. 하위 클래스 생성, 의존성 연결, timer 등록, callback routing 담당   |
| ControlServerClient | Control Server HTTP API 호출 담당                                     |
| TaskManager         | 주문/입고 감지, robot_unit 배정, task 생성, 상태 전이, 실패 처리 담당                 |
| TrafficManager      | zone graph 기반 경로 탐색, 충돌 회피, path 예약/해제, dock 예약 담당                |
| RobotCommandGateway | TaskManager의 task를 ROS2 Action/Service 명령으로 변환해 State Manager에 전송 |
| RobotStateMonitor   | PICKY/COBOT 상태 Topic을 구독하고 TrafficManager/TaskManager에 전달         |

## 7. TaskManager ↔ TrafficManager 내부 인터페이스

TaskManager와 TrafficManager는 HTTP/ROS2가 아니라 같은 프로세스 안의 Python method call로 통신한다.

### PathResult

```
1 @dataclass(frozen=True)
2 class PathResult:
3     ok: bool
4     waypoints: tuple[str, ...] = ()
5     cost: float | None = None
6     reason: str | None = None
```

| 필드        | 의미                                 |
|-----------|------------------------------------|
| ok        | 경로 예약 성공 여부                        |
| waypoints | 시작 zone부터 목적지 zone까지의 zone_name 목록 |
| cost      | 현재 구현 기준 hop 수                     |
| reason    | 실패 사유                              |

### TrafficManager 제공 메서드

| 메서드                                                                           | 목적                                                      |
|-------------------------------------------------------------------------------|---------------------------------------------------------|
| <code>reserve_path(robot_id, task_id, source_zone, target_zone)</code>        | 목적지가 확정된 이동 경로 예약                                       |
| <code>reserve_nearest_from(robot_id, task_id, source_zone, candidates)</code> | 후보 zone 중 가장 가까운 경로 선택 및 예약                             |
| <code>attach_task_id(robot_id, task_id)</code>                                | <code>task_id=None</code> 으로 임시 예약한 path에 DB task_id 연결 |
| <code>reserve_return_home_path(robot_id, task_id, source_zone)</code>         | standby zone 복귀 경로 예약                                   |
| <code>reserve_dock_path(robot_id, task_id, source_zone)</code>                | 충전 dock 선택 및 도킹 경로 예약                                   |



|                                                                              |                                       |
|------------------------------------------------------------------------------|---------------------------------------|
| <code>update_path_progress(robot_id, task_id, current_waypoint_index)</code> | waypoint 통과에 따른 path 점유 해제            |
| <code>release_path(robot_id, task_id)</code>                                 | task 종료 시 path 예약 해제                  |
| <code>notify_state(robot_id, state)</code>                                   | RobotStateMonitor가 수신한 PICKY state 전달 |

#### 상품 후보 선택 호출 예시

```

1 candidates = {
2     "PRODUCT_ZONE_1": 2,
3     "PRODUCT_ZONE_3": 1
4 }
5
6 result = traffic_manager.reserve_nearest_from(
7     robot_id="PICKY1",
8     task_id=None,
9     source_zone="STANDBY_ZONE_1",
10    candidates=candidates,
11 )
12
13 if result.ok:
14     selected_zone = result.waypoints[-1]
```

#### 처리 원칙:

```

1 1. TaskManager가 상품 후보 dict를 만든다.
2 2. TrafficManager는 후보 zone 중 가장 가까운 경로를 예약한다.
3 3. TaskManager는 result.waypoints[-1]로 선택된 zone을 확인한다.
4 4. TaskManager가 해당 상품의 MOVE_TO_PRODUCT task를 생성한다.
5 5. DB에서 발행된 task_id를 attach_task_id()로 TrafficManager 예약에 연결한다.
6 6. task 생성 실패 시 release_path(robot_id, None)으로 임시 예약을 해제한다.
```

#### 픽업 zone 선택 호출 예시

```

1 candidates = {
2     "PICKUP_ZONE_1": 1,
3     "PICKUP_ZONE_2": 1
4 }
5
6 result = traffic_manager.reserve_nearest_from(
7     robot_id="PICKY1",
8     task_id=None,
9     source_zone="PRODUCT_ZONE_3",
10    candidates=candidates,
11 )
12
13 if result.ok:
14     selected_pickup_zone = result.waypoints[-1]
```

#### 처리 원칙:

```

1 1. 모든 상품 상차가 끝난다.
```

```

2 2. TaskManager가 EMPTY pickup slot을 Control Server에서 조회한다.
3 3. EMPTY slot에 대응되는 pickup zone 후보 dict를 만든다.
4 4. TrafficManager가 가장 가까운 pickup zone을 선택한다.
5 5. TaskManager가 선택된 pickup zone과 같은 번호의 pickup slot을 주문에 배정
  한다.
6 6. MOVE_TO_PICKUP / INSPECTION / UNLOAD task를 생성한다.

```

## 8. Control Server ↔ AI Server LLM

Control Server가 관리자 자연어 명령을 구조화된 action으로 변환하기 위해 LLM 담당 모듈 또는 AI Server LLM을 호출한다.

| IF ID   | 호출 시<br>점                 | 송신자               | 수신자                                  | 목적               | 요청 데<br>이터                                      | 응답 데<br>이터                                         | 프로토<br>콜                   |
|---------|---------------------------|-------------------|--------------------------------------|------------------|-------------------------------------------------|----------------------------------------------------|----------------------------|
| IF-L-01 | 관리자<br>자연어<br>명령 입<br>력 시 | Admin<br>UI       | Control<br>Server                    | 자연어<br>명령 전<br>달 | POST<br>/api/<br>admin<br>/llm/<br>messa<br>ges | LLM 처<br>리 결과                                      | HTTP/<br>TCP               |
| IF-L-02 | 자연어<br>파싱 필<br>요 시        | Control<br>Server | AI<br>Server<br>LLM 또<br>는 LLM<br>모듈 | 입고 명<br>령 파싱     | messa<br>ge,<br>context                         | action,<br>product<br>,<br>quantit<br>y,<br>policy | SDK 또<br>는<br>HTTP/<br>TCP |

### LLM action

| action            | 의미             |
|-------------------|----------------|
| STOCKING          | 입고 명령          |
| INVENTORY_SUMMARY | 재고 요약 질의       |
| EXCEPTION_SUMMARY | 예외 요약 질의       |
| CHAT              | 일반 응답 또는 해석 불가 |

### IF-L-02 요청 예시

```

1 {

```

```

2  "message": "콜라 5개 입고해줘",
3  "context": {
4    "products": [
5      {
6        "product_id": 1,
7        "name": "콜라"
8      },
9      {
10       "product_id": 2,
11       "name": "생수"
12     }
13   ],
14   "low_stock_count": 2,
15   "unresolved_exception_count": 0
16 }
17 }

```

## IF-L-02 응답 예시

```

1  {
2    "action": "STOCKING",
3    "product_id": 1,
4    "product_name": "콜라",
5    "requested_quantity": 5,
6    "stocking_policy": "REQUESTED_QUANTITY",
7    "message": "콜라 5개 입고 요청으로 해석했습니다."
8  }

```

## 처리 원칙:

1. LLM 담당 모듈은 task를 직접 만들지 않는다.
2. Control Server는 LLM 결과를 바탕으로 stocking\_item을 생성한다.
3. Fleet Manager는 REQUESTED stocking\_item을 polling으로 감지한다.
4. TaskManager가 입고 task 4개를 생성한다.

## 9. PICKY/COBOT State Manager ↔ AI Server Vision

Vision 요청은 이미지/프레임이 발생하는 로봇 쪽에서 직접 수행한다.

Control Server는 이미지/영상 데이터를 중계하지 않는다.

| IF ID   | 호출 시<br>점                            | 송신자                           | 수신자                    | 목적                         | 요청 데<br>이터                                   | 응답 데<br>이터                                                 | 프로토<br>콜                          |
|---------|--------------------------------------|-------------------------------|------------------------|----------------------------|----------------------------------------------|------------------------------------------------------------|-----------------------------------|
| IF-V-01 | SORTI<br>NG_AN<br>D_LOA<br>D 수행<br>전 | COBOT<br>State<br>Manag<br>er | AI<br>Server<br>Vision | 주문 상<br>품 인식<br>및 좌표<br>추정 | task_id,<br>product<br>,<br>camera<br>_frame | detecte<br>d,<br>coordin<br>ates_6d<br>,<br>confide<br>nce | HTTP/<br>gRPC/<br>ROS2<br>Service |

|         |                    |                              |                  |                  |                                      |                             |                              |
|---------|--------------------|------------------------------|------------------|------------------|--------------------------------------|-----------------------------|------------------------------|
| IF-V-02 | INSPECTION 수행 중    | COBOT State Manager          | AI Server Vision | 주문 상품 검수         | task_id, order_items, camera_frame   | missing, excess, wrong      | HTTP/gRPC/ROS2 Service       |
| IF-V-03 | STOCKING_PICK 수행 중 | COBOT State Manager          | AI Server Vision | 입고 상품 인식 및 수량 감지 | task_id, stocking_item, camera_frame | detected_quantity, objects  | HTTP/gRPC/ROS2 Service       |
| IF-V-04 | 이동/작업 중 사람 감지 필요 시 | Vision Node 또는 State Manager | AI Server Vision | 사람 감지            | robot, camera_frame, zone            | person_detected, confidence | HTTP/gRPC/ROS2 Topic/Service |

#### Vision 결과 처리 원칙

| 항목       | 원칙                                                 |
|----------|----------------------------------------------------|
| 상품 인식 성공 | COBOT State Manager가 작업을 진행하고 Fleet Manager에 결과 보고 |
| 상품 인식 실패 | Fleet Manager가 재시도 또는 exception_log 기록             |
| 검수 실패    | Fleet Manager가 INSPECTION_FAIL 예외 보고               |
| 사람 감지    | Fleet Manager가 HUMAN_DETECTED 예외 보고 및 필요 시 긴급 정지   |
| 이미지 중계   | Control Server는 이미지/영상 프레임을 중계하지 않음                |

10. Fleet Manager ↔ ROS2 / State Manager

Fleet Manager는 RobotCommandGateway를 통해 PICKY/COBOT State Manager에 실제 작업을 지시한다.

State Manager는 로봇 컨트롤러와 통신하고, 결과를 Fleet Manager에 반환한다.

ROS2 인터페이스 분류

| 용도                      | 방식                   |
|-------------------------|----------------------|
| PICKY 이동/복귀/도킹 명령       | ROS2 Action          |
| COBOT 선별/상차/검수/하차/입고 명령 | ROS2 Action          |
| 로봇 상태 보고                | ROS2 Topic           |
| 작업 진행 feedback          | ROS2 Action feedback |
| 작업 완료/실패 result         | ROS2 Action result   |
| 긴급 정지/재개                | ROS2 Service         |

Fleet Manager ↔ PICKY State Manager

| IF ID   | 호출 시점                | 송신자           | 수신자                 | 목적          | ROS2 이름                                    | ROS2 타입                         | Control Server 변환                         |
|---------|----------------------|---------------|---------------------|-------------|--------------------------------------------|---------------------------------|-------------------------------------------|
| IF-R-01 | MOVE_TO_PRODUCT 실행 시 | Fleet Manager | PICKY State Manager | 상품 구역 이동 명령 | <code>/picky_namespace/move_command</code> | <code>MoveCommand.action</code> | 성공 시 task SUCCESS, 실패 시 NAVIGATION_FAILED |
| IF-R-02 | MOVE_TO_PICKUP 실행 시  | Fleet Manager | PICKY State Manager | 픽업존 이동 명령   | <code>/picky_namespace/move</code>         | <code>MoveCommand.action</code> | 성공 시 task SUCCESS, 실패 시                   |

|         |                                 |                      |                               |                        |                                            |                                |                                                                    |
|---------|---------------------------------|----------------------|-------------------------------|------------------------|--------------------------------------------|--------------------------------|--------------------------------------------------------------------|
|         |                                 |                      |                               |                        | e_com<br>mand                              |                                | 패 시<br>NAVIG<br>ATION<br>_FAILE<br>D                               |
| IF-R-03 | MOVE_<br>TO_ST<br>OCK 실행 시      | Fleet<br>Manag<br>er | PICKY<br>State<br>Manag<br>er | 입고존<br>이동 명<br>령       | / {pic<br>ky_ns<br>} /mov<br>e_com<br>mand | MoveC<br>omman<br>d.act<br>ion | 성공 시<br>task<br>SUCCE<br>SS, 실패 시<br>NAVIG<br>ATION<br>_FAILE<br>D |
| IF-R-04 | MOVE_<br>TO_ST<br>ORAGE<br>실행 시 | Fleet<br>Manag<br>er | PICKY<br>State<br>Manag<br>er | 적재 위<br>치 이동<br>명령     | / {pic<br>ky_ns<br>} /mov<br>e_com<br>mand | MoveC<br>omman<br>d.act<br>ion | 성공 시<br>task<br>SUCCE<br>SS, 실패 시<br>NAVIG<br>ATION<br>_FAILE<br>D |
| IF-R-05 | RETUR<br>N_HO<br>ME 실행 시        | Fleet<br>Manag<br>er | PICKY<br>State<br>Manag<br>er | standb<br>y zone<br>복귀 | / {pic<br>ky_ns<br>} /mov<br>e_com<br>mand | MoveC<br>omman<br>d.act<br>ion | 성공 시<br>task<br>SUCCE<br>SS                                        |
| IF-R-06 | DOCK_<br>IN 실행<br>시             | Fleet<br>Manag<br>er | PICKY<br>State<br>Manag<br>er | 충전 도<br>크 정밀<br>진입     | / {pic<br>ky_ns<br>} /mov<br>e_com<br>mand | MoveC<br>omman<br>d.act<br>ion | 성공 시<br>picky_s<br>tate=C<br>HARGI<br>NG                           |
| IF-R-07 | PICKY<br>상태 변                   | PICKY<br>State       | Fleet<br>Manag                | PICKY<br>상태 보          | / {pic<br>ky_ns                            | std_m<br>sgs/m                 | Traffic<br>Manag                                                   |

|         |                      |                               |                               |                  |                                                 |                                                  |                                          |
|---------|----------------------|-------------------------------|-------------------------------|------------------|-------------------------------------------------|--------------------------------------------------|------------------------------------------|
|         | 경 시                  | Manag<br>er                   | er                            | 고                | }/pic<br>ky_st<br>ate                           | sg/St<br>ring                                    | er 상태<br>갱신 및<br>robot<br>상태 보<br>고      |
| IF-R-08 | PICKY<br>배터리<br>변경 시 | PICKY<br>State<br>Manag<br>er | Fleet<br>Manag<br>er          | 배터리<br>상태 보<br>고 | / {pic<br>ky_ns<br>}/bat<br>tery                | senso<br>r_msg<br>s/msg<br>/Batt<br>erySt<br>ate | PATCH<br>robot<br>상태                     |
| IF-R-09 | 긴급 정<br>지/재개<br>시    | Fleet<br>Manag<br>er          | PICKY<br>State<br>Manag<br>er | 긴급 정<br>지/해제     | / {pic<br>ky_ns<br>}/eme<br>rgenc<br>y_sto<br>p | std_s<br>rvs/s<br>rv/Se<br>tBoo<br>l             | Admin<br>emerge<br>ncy/res<br>ume 변<br>환 |

#### Fleet Manager ↔ COBOT State Manager

| IF ID   | 호출 시<br>점                            | 송신자                  | 수신자                           | 목적                               | ROS2<br>이름                                | ROS2<br>타입                     | Control<br>Server<br>변환                             |
|---------|--------------------------------------|----------------------|-------------------------------|----------------------------------|-------------------------------------------|--------------------------------|-----------------------------------------------------|
| IF-R-10 | SORTI<br>NG_AN<br>D_LOA<br>D 실행<br>시 | Fleet<br>Manag<br>er | COBOT<br>State<br>Manag<br>er | 주문 상<br>품 선별<br>및<br>PICKY<br>상차 | / {cob<br>ot_ns<br>}/exe<br>cute_<br>task | Execu<br>teTas<br>k.act<br>ion | 성공 시<br>order_it<br>em.stat<br>us=SO<br>RTED        |
| IF-R-11 | INSPE<br>CTION<br>실행 시               | Fleet<br>Manag<br>er | COBOT<br>State<br>Manag<br>er | 주문 상<br>품 검수                     | / {cob<br>ot_ns<br>}/exe<br>cute_<br>task | Execu<br>teTas<br>k.act<br>ion | 성공 시<br>order_it<br>em.stat<br>us=INS<br>PECTE<br>D |

|         |                     |                     |                     |                     |                        |                     |                                               |
|---------|---------------------|---------------------|---------------------|---------------------|------------------------|---------------------|-----------------------------------------------|
| IF-R-12 | UNLOAD 실행 시         | Fleet Manager       | COBOT State Manager | 픽업 슬롯 하차            | /cobots/execute_task   | ExecuteTask.action  | 성공 시 pickup_slot=OCCUPIED, order=PICKUP_READY |
| IF-R-13 | STOCKING_PICK 실행 시  | Fleet Manager       | COBOT State Manager | 입고 상품 선별 및 PICKY 상차 | /cobots/execute_task   | ExecuteTask.action  | 성공 시 detected_quantity 기록                     |
| IF-R-14 | STOCKING_PLACE 실행 시 | Fleet Manager       | COBOT State Manager | 입고 상품 적재            | /cobots/execute_task   | ExecuteTask.action  | 성공 시 stocking complete                        |
| IF-R-15 | COBOT 상태 변경 시       | COBOT State Manager | Fleet Manager       | COBOT 상태 보고         | /cobots/cobots_state   | std_msgs/Msg/String | robot 상태 보고                                   |
| IF-R-16 | 긴급 중지/재개 시          | Fleet Manager       | COBOT State Manager | 긴급 중지/해제            | /cobots/emergency_stop | std_srvs/SetBool    | Admin emergency/resume 변환                     |

## 11. ROS2 PICKY Interface

적용 로봇



| robot_name | ROS namespace |
|------------|---------------|
| PICKY1     | /picky1       |
| PICKY2     | /picky2       |

담당 task\_type

| task_type       | 설명               |
|-----------------|------------------|
| MOVE_TO_PRODUCT | 주문 상품 보관 구역으로 이동 |
| MOVE_TO_PICKUP  | 픽업존으로 이동         |
| MOVE_TO_STOCK   | 입고존으로 이동         |
| MOVE_TO_STORAGE | 적재 위치 이동         |
| RETURN_HOME     | standby zone 복귀  |
| DOCK_IN         | 충전 도크 정밀 진입      |
| CHARGE          | 충전 상태 진입/유지      |

PICKY ROS2 Interface

| IF ID   | 호출 시<br>점       | 송신자                 | 수신자                 | 목적             | ROS2 이<br>름            | ROS2 타<br>입                  |
|---------|-----------------|---------------------|---------------------|----------------|------------------------|------------------------------|
| IF-P-01 | 이동<br>task 실행 시 | Fleet Manager       | PICKY State Manager | waypoint 이동 명령 | /picky_ns/move_command | MoveCommand.action           |
| IF-P-02 | 상태 변경 시         | PICKY State Manager | Fleet Manager       | 상태 보고          | /picky_ns/picky_state  | std_msgs/String              |
| IF-P-03 | 배터리 변경 시        | PICKY State Manager | Fleet Manager       | 배터리 보고         | /picky_ns/             | sensor_msgs/msg/BatteryState |

|         |            |               |                     |          |                             |                     |
|---------|------------|---------------|---------------------|----------|-----------------------------|---------------------|
|         |            |               |                     |          | battery                     | batteryState        |
| IF-P-04 | 긴급 정지/해제 시 | Fleet Manager | PICKY State Manager | 긴급 정지/해제 | /pick_y_ns}/emergency_state | std_srvs/SetBoolean |

## MoveCommand.action

```

1 # Goal
2 string task_type
3 geometry_msgs/PoseStamped[] waypoints
4 ---
5 # Result
6 bool success
7 string message
8 ---
9 # Feedback
10 uint8 current_waypoint_index
11 uint8 total_waypoints

```

## MoveCommand goal 예시

```

1 {
2   "task_type": "MOVE_TO_PRODUCT",
3   "waypoints": [
4     {
5       "header": {
6         "frame_id": "map"
7       },
8       "pose": {
9         "position": {
10          "x": 1.2,
11          "y": 3.4,
12          "z": 0.0
13        },
14        "orientation": {
15          "z": 0.0,
16          "w": 1.0
17        }
18      }
19    }
20  ]
21 }

```

## 처리 원칙:

1. RobotCommandGateway가 task\_id와 action goal을 내부적으로 매핑한다.
2. MoveCommand.action goal 자체에는 task\_id가 없다.
3. action feedback의 current\_waypoint\_index는 TrafficManager.update\_path\_progress()에 전달한다.
4. action result success=true이면 task SUCCESS로 보고한다.
5. action result success=false이면 task FAILED 및 exception\_log를 기록한다.

12. ROS2 COBOT Interface

적용 로봇

| robot_name | ROS namespace |
|------------|---------------|
| COBOT1     | /cobot1       |
| COBOT2     | /cobot2       |

담당 task\_type

| task_type        | 설명                  |
|------------------|---------------------|
| SORTING_AND_LOAD | 주문 상품 선별 및 PICKY 상차 |
| INSPECTION       | 주문 상품 검수            |
| UNLOAD           | 픽업 슬롯 하차            |
| STOCKING_PICK    | 입고 상품 선별 및 PICKY 상차 |
| STOCKING_PLACE   | 입고 상품 적재            |

COBOT ROS2 Interface

| IF ID    | 호출 시<br>점          | 송신자                       | 수신자                       | 목적                 | ROS2 이<br>름                | ROS2 타<br>입                 |
|----------|--------------------|---------------------------|---------------------------|--------------------|----------------------------|-----------------------------|
| IF-CB-01 | COBOT<br>task 실행 시 | Fleet<br>Manager          | COBOT<br>State<br>Manager | COBOT<br>작업 명<br>령 | /cobot_ns/<br>execute_task | ExecuteTask.action          |
| IF-CB-02 | 상태 변경 시            | COBOT<br>State<br>Manager | Fleet<br>Manager          | COBOT<br>상태 보<br>고 | /cobot_ns/<br>cobot_state  | std_msgs/msg/<br>String     |
| IF-CB-03 | 긴급 정지/해제<br>시      | Fleet<br>Manager          | COBOT<br>State<br>Manager | 긴급 정지/해제           | /cobot_ns/<br>emergency    | std_srvs/srv/<br>SetBoolean |

|  |  |  |  |  |        |  |
|--|--|--|--|--|--------|--|
|  |  |  |  |  | ncy_st |  |
|  |  |  |  |  | op     |  |

## ExecuteTask.action

```

1  # Goal
2  int32 task_id
3  int32 order_id
4  int32 order_item_id
5  int32 stocking_item_id
6  string task_type
7  int32 source_zone_id
8  int32 target_zone_id
9  int32 product_id
10 string product_name
11 int32 quantity
12 int32 requested_quantity
13 int32 detected_quantity
14 int32 pickup_slot_id
15 ---
16 # Result
17 bool success
18 string status
19 string result_message
20 int32 detected_quantity
21 int32 stock_delta
22 ---
23 # Feedback
24 string status
25 string message
26 float32 progress

```

## ExecuteTask goal 예시 - SORTING\_AND\_LOAD

```

1  {
2    "task_id": 102,
3    "order_id": 7,
4    "order_item_id": 22,
5    "stocking_item_id": 0,
6    "task_type": "SORTING_AND_LOAD",
7    "source_zone_id": 12,
8    "target_zone_id": 6,
9    "product_id": 3,
10   "product_name": "과자",
11   "quantity": 1,
12   "requested_quantity": 0,
13   "detected_quantity": 0,
14   "pickup_slot_id": 0
15 }

```

## ExecuteTask goal 예시 - STOCKING\_PICK

```

1  {
2    "task_id": 203,
3    "order_id": 0,
4    "order_item_id": 0,
5    "stocking_item_id": 1,
6    "task_type": "STOCKING_PICK",
7    "source_zone_id": 4,
8    "target_zone_id": 4,
9    "product_id": 1,
10   "product_name": "콜라",

```

```

11  "quantity": 0,
12  "requested_quantity": 0,
13  "detected_quantity": 0,
14  "pickup_slot_id": 0
15  }

```

처리 원칙:

- 1 1. COBOT State Manager는 ExecuteTask.action goal을 받아 작업을 수행한다.
- 2 2. Vision 요청이 필요하면 COBOT State Manager가 AI Vision Server에 직접 요청한다.
- 3 3. action result success=true이면 Fleet Manager가 task SUCCESS로 보고한다.
- 4 4. action result success=false이면 Fleet Manager가 task FAILED 및 exception\_log를 기록한다.
- 5 5. STOCKING\_PICK/STOCKING\_PLACE 결과의 detected\_quantity, stock\_delta는 입고 완료 처리에 사용한다.

### 13. Emergency Stop Service

| 항목           | 값                                |
|--------------|----------------------------------|
| Service name | /robot_namespace/emergency_stop  |
| Type         | std_srvs/srv/SetBool             |
| Request      | {data: bool}                     |
| Response     | {success: bool, message: string} |
| data=true    | 긴급 정지                            |
| data=false   | 긴급 정지 해제 요청                      |

처리 원칙:

- 1 1. Admin UI가 emergency-stop/resume 요청을 Control Server에 보낸다.
- 2 2. Fleet Manager가 Admin 상태 또는 별도 API를 통해 emergency 상태를 감지한다.
- 3 3. Fleet Manager가 각 State Manager의 emergency\_stop service를 호출한다.
- 4 4. 실패한 로봇은 exception\_log에 기록한다.

### 14. Robot ID / Namespace

| robot_id | robot_name | robot_type | robot_unit | ROS namespace | 미니맵 표시 |
|----------|------------|------------|------------|---------------|--------|
|----------|------------|------------|------------|---------------|--------|

|   |        |       |                  |         |        |
|---|--------|-------|------------------|---------|--------|
| 1 | PICKY1 | PICKY | PICKY_UN<br>IT_1 | /picky1 | 표시     |
| 2 | COBOT1 | COBOT | PICKY_UN<br>IT_1 | /cobot1 | 표시 안 함 |
| 3 | PICKY2 | PICKY | PICKY_UN<br>IT_2 | /picky2 | 표시     |
| 4 | COBOT2 | COBOT | PICKY_UN<br>IT_2 | /cobot2 | 표시 안 함 |

## 15. Zone / Slot 기준

| 구분       | 의미                           |
|----------|------------------------------|
| *_ZONE_* | PICKY가 이동하거나 정차하는 위치         |
| *_SLOT_* | COBOT이 물건을 집거나 내려놓는 실제 작업 위치 |

### 주요 매핑

| PICKY 정차 위치    | COBOT 작업 위치    |
|----------------|----------------|
| PRODUCT_ZONE_1 | PRODUCT_SLOT_1 |
| PRODUCT_ZONE_2 | PRODUCT_SLOT_2 |
| PRODUCT_ZONE_3 | PRODUCT_SLOT_3 |
| PRODUCT_ZONE_4 | PRODUCT_SLOT_4 |
| PRODUCT_ZONE_5 | PRODUCT_SLOT_5 |
| PRODUCT_ZONE_6 | PRODUCT_SLOT_6 |
| PICKUP_ZONE_1  | PICKUP_SLOT_1  |
| PICKUP_ZONE_2  | PICKUP_SLOT_2  |
| STOCK_ZONE     | STOCK_SLOT     |

처리 원칙:

1. PICKY 이동 task의 target은 ZONE을 사용한다.
2. COBOT 작업 task의 source/target은 SLOT 또는 작업 기준 위치를 사용한다.
3. 상품 보관 위치는 PRODUCT\_SLOT\_\*이다.
4. PICKY가 해당 상품 앞에 정차하는 위치는 PRODUCT\_ZONE\_\*이다.
5. pickup slot은 상품 상차가 모두 끝난 뒤 MOVE\_TO\_PICKUP 직전에 배정한다.

## 16. 주문 task 흐름

주문 task는 상품별 이동/상차를 반복한 뒤 픽업존 이동/검수/하차로 마무리한다.

```
1 상품 1:
2   MOVE_TO_PRODUCT
3   SORTING_AND_LOAD
4
5 상품 2:
6   MOVE_TO_PRODUCT
7   SORTING_AND_LOAD
8
9 ...
10
11 모든 상품 상차 완료 후:
12   MOVE_TO_PICKUP
13   INSPECTION
14   UNLOAD
```

처리 원칙:

1. TaskManager가 주문 item 목록을 dict로 관리한다.
2. TaskManager가 남은 상품 후보를 {PRODUCT\_ZONE\_N: quantity} 형태로 TrafficManager에 전달한다.
3. TrafficManager가 가장 가까운 상품 zone을 선택하고 path를 예약한다.
4. TaskManager가 선택된 상품에 대해 MOVE\_TO\_PRODUCT/SORTING\_AND\_LOAD task를 생성한다.
5. 모든 상품 상차가 끝나면 EMPTY pickup slot을 조회한다.
6. TaskManager가 pickup zone 후보를 TrafficManager에 전달한다.
7. TrafficManager가 가장 가까운 pickup zone을 선택한다.
8. TaskManager가 같은 번호의 pickup slot을 주문에 배정한다.
9. MOVE\_TO\_PICKUP/INSPECTION/UNLOAD task를 생성한다.

## 17. 입고 task 흐름

입고 task는 LLM이 만든 stocking\_item을 기준으로 TaskManager가 생성한다.

```
1 MOVE_TO_STOCK
2 STOCKING_PICK
3 MOVE_TO_STORAGE
4 STOCKING_PLACE
```

처리 원칙:

1. LLM 담당 모듈은 자연어를 해석해 Control Server에 stocking\_item을 생성한다.
2. Fleet Manager는 REQUESTED stocking\_item을 polling으로 감지한다.
3. TaskManager가 robot\_unit을 배정한다.
4. TaskManager가 입고 task 4개를 생성한다.
5. STOCKING\_PLACE 성공 후 POST /api/fleet/stocking/complete를 호출한다.

## 18. 작업 종료 후 복귀/도킹/충전 흐름

주문 또는 입고 작업이 끝난 뒤 TaskManager가 다음 작업 여부와 배터리 상태를 보고 복귀/도킹/충전을 결정한다.

```

1 UNLOAD 또는 STOCKING_PLACE 완료
2   -> 대기 주문/입고가 있으면 다음 작업 수행
3   -> 대기 작업이 없으면 RETURN_HOME
4   -> standby zone 도착
5   -> 필요하면 DOCK_IN
6   -> 도킹 성공 후 CHARGE 또는 robot_status=CHARGING

```

| task_type   | 의미              | TrafficManager 호출          |
|-------------|-----------------|----------------------------|
| RETURN_HOME | standby zone 복귀 | reserve_return_home_path() |
| DOCK_IN     | 충전 도크 정밀 진입     | reserve_dock_path()        |
| CHARGE      | 충전 상태 진입/유지     | 경로 예약 없음                   |

처리 원칙:

```

1 1. RETURN_HOME은 standby zone 도착까지만 담당한다.
2 2. DOCK_IN은 충전 dock 선택과 정밀 도킹을 담당한다.
3 3. CHARGE는 충전 상태 진입/유지로 본다.
4 4. DOCK_IN 실패 시 dock 예약 해제 정책을 TrafficManager와 맞춘다.

```

## 19. ENUM 요약

order\_status

```

1 ORDER_RECEIVED
2 ORDER_WAIT
3 SORTING
4 DELIVERING
5 INSPECTING
6 PICKUP_READY
7 COMPLETED
8 ERROR

```

order\_item\_status

```

1 WAITING
2 SORTED
3 INSPECTED
4 MISSING
5 EXCESS

```



**pickup\_slot\_status**

|   |          |
|---|----------|
| 1 | EMPTY    |
| 2 | RESERVED |
| 3 | OCCUPIED |
| 4 | BLOCKED  |

**robot\_type**

|   |       |
|---|-------|
| 1 | PICKY |
| 2 | COBOT |

**robot\_status**

|   |                |
|---|----------------|
| 1 | OFFLINE        |
| 2 | IDLE           |
| 3 | BUSY           |
| 4 | CHARGING       |
| 5 | EMERGENCY_STOP |
| 6 | ERROR          |

**picky\_state**

|    |                   |
|----|-------------------|
| 1  | CHARGING          |
| 2  | STANDBY           |
| 3  | MOVING_TO_PRODUCT |
| 4  | WAITING_FOR_COBOT |
| 5  | MOVING_TO_PICKUP  |
| 6  | MOVING_TO_STOCK   |
| 7  | MOVING_TO_STORAGE |
| 8  | RETURNING         |
| 9  | DOCKING           |
| 10 | ERROR_RECOVERY    |

**cobot\_state**

|    |                  |
|----|------------------|
| 1  | STANDBY          |
| 2  | SORTING          |
| 3  | LOADING          |
| 4  | INSPECTING       |
| 5  | UNLOADING        |
| 6  | STOCKING_SORTING |
| 7  | STOCKING_LOADING |
| 8  | STOCKING_PLACING |
| 9  | STOWING_ARM      |
| 10 | SAFETY_STOPPED   |

**task\_type**

|    |                  |
|----|------------------|
| 1  | MOVE_TO_PRODUCT  |
| 2  | SORTING_AND_LOAD |
| 3  | MOVE_TO_PICKUP   |
| 4  | INSPECTION       |
| 5  | UNLOAD           |
| 6  | MOVE_TO_STOCK    |
| 7  | STOCKING_PICK    |
| 8  | MOVE_TO_STORAGE  |
| 9  | STOCKING_PLACE   |
| 10 | RETURN_HOME      |
| 11 | DOCK_IN          |

**task\_status**

```

1 QUEUED
2 ASSIGNED
3 RUNNING
4 PAUSED
5 SUCCESS
6 FAILED
7 CANCELLED

```

**stocking\_policy**

```

1 REQUESTED_QUANTITY
2 ALL_DETECTED

```

**stocking\_item\_status**

```

1 REQUESTED
2 ASSIGNED
3 IN_PROGRESS
4 COMPLETED
5 FAILED
6 CANCELLED

```

**exception\_type**

```

1 OBSTACLE_DETECTED
2 LOW_BATTERY
3 NAVIGATION_FAILED
4 HARDWARE_ERROR
5 TIMEOUT
6 SORTING_FAIL
7 INSPECTION_FAIL
8 HUMAN_DETECTED
9 SYSTEM_ERROR

```

**llm\_action**

```

1 STOCKING
2 INVENTORY_SUMMARY
3 EXCEPTION_SUMMARY
4 CHAT

```

**20. 금지/주의 항목**

| 항목                                          | 이유                                            |
|---------------------------------------------|-----------------------------------------------|
| PICKY/COBOT이 Control Server task를 직접 조회     | task 생성/배정/상태 전이는 Fleet Manager 책임이다.         |
| Control Server가 직접 PICKY/COBOT에 이동/작업 명령 전송 | 로봇 제어 책임은 Fleet Manager와 State Manager가 담당한다. |

|                                                  |                                                                            |
|--------------------------------------------------|----------------------------------------------------------------------------|
| Control Server가 이미지/영상 중계                        | 이미지/영상은 Vision 요청 주체가 AI Vision Server에 직접 보낸다.                            |
| Control Server가 주문 생성 시 task 생성                  | task 생성은 Fleet Manager TaskManager 책임이다.                                   |
| Control Server가 Fleet Manager에 주문 생성 push 필수로 의존 | 주문/입고 감지는 Fleet Manager polling 기준이다.                                      |
| task_type=PATROL                                 | 순찰 시나리오 제거로 사용하지 않는다.                                                      |
| exception_type=FIRE_DETECTED                     | 순찰/화재 감지 시나리오 제거로 사용하지 않는다.                                                |
| task_type=DELIVERY                               | 운반 흐름은 MOVE_TO_PRODUCT, MOVE_TO_PICKUP 등으로 분리한다.                           |
| task_type=PICK_AND_LOAD                          | 최신 명칭은 SORTING_AND_LOAD 이다.                                                |
| task.payload_json 사용                             | 입고 데이터는 stocking_item, 주문 상품 데이터는 order_item으로 관리한다.                       |
| robot_status에 세부 작업 상태를 모두 넣기                    | 큰 상태는 robot_status, 세부 상태는 picky_state/cobot_state로 분리한다.                  |
| COBOT을 미니맵 이동 객체로 표시                             | 미니맵에는 robot_type=PICKY만 표시한다.                                              |
| 입고 수량을 단일 quantity만으로 관리                         | requested_quantity, detected_quantity, stocking_policy, stock_delta로 구분한다. |
| pickup slot을 주문 초기에 고정 배정                        | 모든 상품 상차 후 MOVE_TO_PICKUP 직전에 TrafficManager가 선택한 pickup zone에 맞춰 배정한다.    |